INTRODUCTION TO EMBEDDED SYSTEMS AND IOTS

Lab Name: Embedded Light Follow System

Lab Number: 2

Date: 10/23/26

Student Name: Perry Ferrara

Student ID: 703119415

Group Name: None

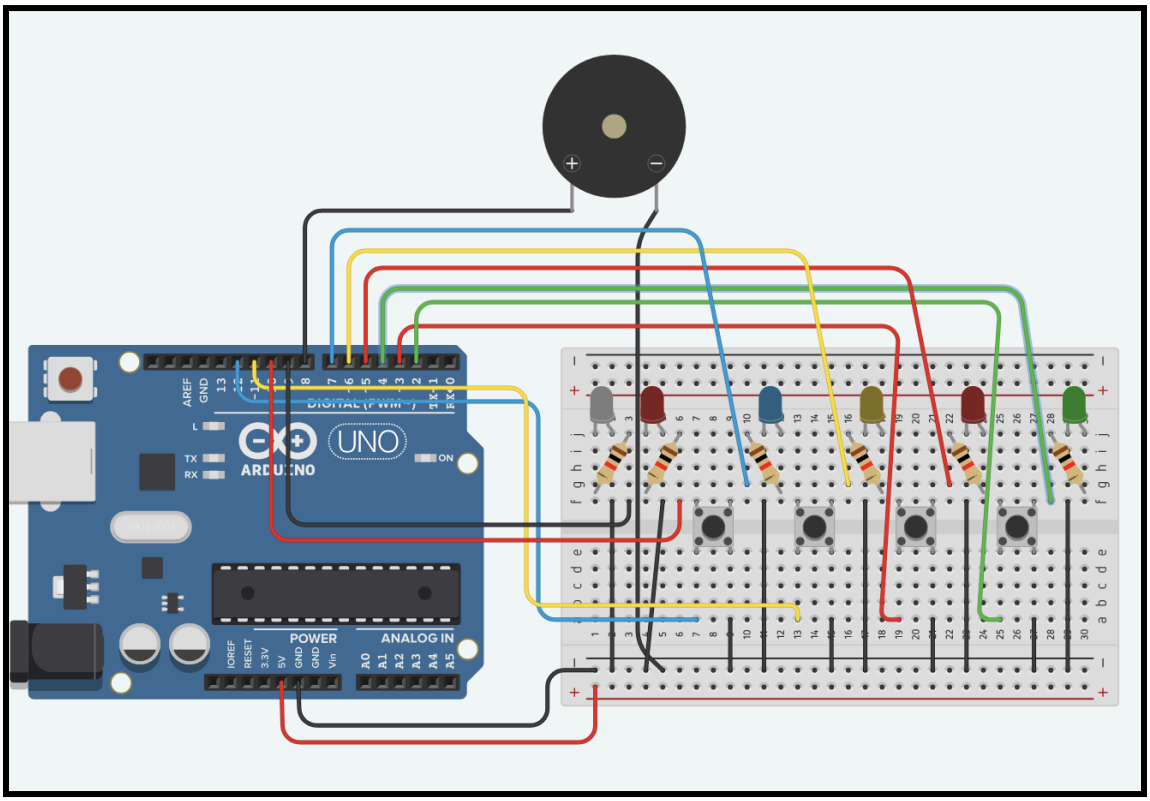
INTRODUCTION OF THE LAB AND REPORT

This second assignment focuses on creating an embedded memory game similar to Simon using an Arduino, LEDs, push buttons, and a passive buzzer. The program presents an incrementally increasing sequence of colored lights, with a different sound assigned to each color. The player must repeat the sequence using the corresponding buttons. Each successful round adds one color to the existing sequence, up to a maximum length of 20. Separate white and red status LEDs indicate correct and incorrect rounds, accompanied by distinct buzzer chimes for each.

MATERIALS, DEVICES, AND INSTRUMENTS

- Arduino Uno Rev3 Board
- Mini Breadboard
- Resistors
    - 6x – 220Ω
- 4x Pushbutton
- LED Lights (Green, White, 2x Red, Yellow, Blue)
- Passive Buzzer
- Wires

SCHEMATICS



LAB METHODS AND PROCEDURE

VS Code represents the four colors with integers from 0 to 3. Arrays associate each color number with an LED pin, a button pin, and a musical note from pitches.h, while a separate array stores the game sequence. The initial program uses an analog reading from pin A0 as a seed for the pseudorandom-number generator.

1. At the beginning of a round, add one random color to the stored sequence. Display the entire sequence with each LED and its corresponding tone active for 500 milliseconds, followed by a 250 millisecond pause.

2. Wait for the player to press a button. Confirm the press after a 25 millisecond debounce delay, provide the matching light and sound, and wait until all buttons have been released for 25 milliseconds.

3. Compare each selected color with the corresponding sequence entry. A mismatch calls the incorrect-round feedback function, resetting the sequence length to one. Matching the entire sequence calls the correct-round function, increasing the length by one.
4. Test each button separately, then test correct and incorrect sequences. Verify that held buttons do not count repeatedly and that the program restarts after an error. At length 20, verify the completion message and restart behavior.

TROUBLE IN THE LAB AND HOW DID YOU SOLVE IT

The program was developed in phases, beginning with random-number generation (which was performed in class) and then adding and testing LED output, sound, round-status indicators, and button input. One programming consideration was preventing one button press from being counted more than once. The button-reading function confirms a press after a short delay, and a separate release function waits until all buttons remain released for 25 milliseconds before continuing. The release function uses a while loop, and resets its timer whenever any button is still pressed.

Another consideration was deciding when to generate new numbers. The initial test code filled the entire sequence array during every execution of loop(), but for the memory game, the previous colors need to remain unchanged after a successful round. The revised logic adds only one random number at a time, preserving the existing sequence while extending it for the next round.

The sound output was organized into separate functions so the same passive buzzer could provide both color notes and round feedback. The correct-round function uses an ascending C5, E5, G5 chime with the green status LED. The incorrect-round function flashes the red status LED twice while playing G3 followed by C3. Determining the tune for these chimes required online assistance as to find something pleasing to the ear.

RESULTS AND CONCLUSION

The program developed for this assignment combines a stored random sequence, four color and sound outputs, four button inputs, and separate round-feedback functions. Its intended behavior is to retain the sequence after each correct round, extend it by one color, and restart after

an incorrect response. The game is configured for a maximum sequence length of 20. This project illustrates how digital inputs and outputs can be coordinated through arrays and reusable functions. Developing the features in stages makes it easier to isolate problems in wiring, input detection, and game logic.